

Deep Networks

From Shallow to Feedforward, and Universal Approximation

Fundamentals of Deep Learning · Class 7

5 August

Outline

Composing Networks

The Deep Network

Universal Approximation

Depth versus Width

What Deep Networks Represent

Composing Networks

Notation used throughout this course

$\mathbf{x} \in \mathbb{R}^D$	input / features	L	depth : number of hidden layers
$y, \hat{y}$	target, model prediction	M	width : units per hidden layer
$\mathbf{z}^{(l)}$	units of hidden layer l	D, K	inputs, outputs
$a_j^{(l)}$	their pre-activations	$\phi_j(\mathbf{x})$	fixed basis function
$h(\cdot)$	activation, ReLU throughout	$\mathcal{K}$	a compact subset of $\mathbb{R}^D$
$\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$	weights and biases of layer l	$f \circ g$	composition, f after g

A shallow network is $L = 1$. Today $L > 1$: the same block, **composed**.

Where the shallow network left us

- A shallow network is a linear read-out over **learned features**
 $z_j = h(\mathbf{w}_j^{(1)\top} \mathbf{x} + w_{j0}^{(1)})$ (Classes 5–6)
- With enough units it is a **universal approximator** — we proved this constructively for $D = 1$ in Class 5
- But the width needed grows like $(L_f/\varepsilon)^D$: the curse of dimensionality
- Question for today: what if we feed the output of one network into the **input of another**?

Stacking shallow networks gives a **deep network**: the same building block, composed — and far more efficient.

An intuition: solve a big problem in stages

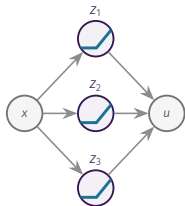
- A shallow network must map raw inputs to the answer in **one leap** — every hidden unit works directly on $\mathbf{x}$
- A deep network works in **stages**: layer 1 extracts simple features, layer 2 combines them into richer ones, and so on
- Reading is the analogy: **strokes** $\rightarrow$ **letters** $\rightarrow$ **words** $\rightarrow$ **meaning**. Nobody jumps from ink to meaning in one step
- Each stage is *easy* because it builds on the stage before — the difficulty is **spread across depth**

SHALLOW one leap: raw input $\rightarrow$ answer

DEEP staged: input $\rightarrow$ features $\rightarrow$ features of features $\rightarrow$ answer

Two networks to work with

Network 1



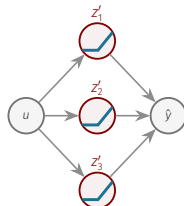
$$z_1 = h(x + 1)$$

$$z_2 = h(x + 0.5)$$

$$z_3 = h(x - 0.5)$$

$$u = f_1(x) = -1 + 4z_1 - 6z_2 + 6z_3$$

Network 2



$$z'_1 = h(u + 1)$$

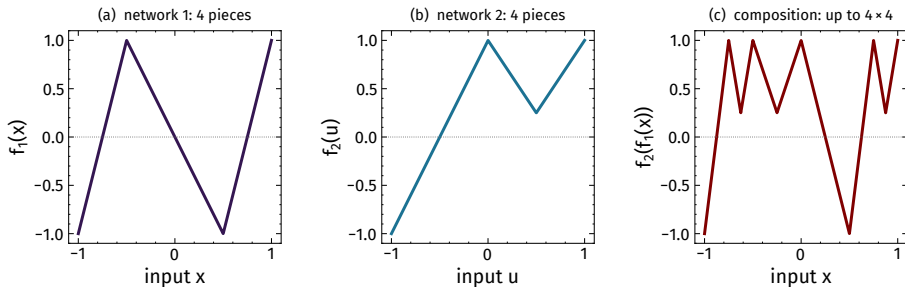
$$z'_2 = h(u)$$

$$z'_3 = h(u - 0.5)$$

$$f_2(u) = -1 + 2z'_1 - 3.5z'_2 + 3z'_3$$

Each is piecewise linear with **four pieces**; f_1 traces $-1 \rightarrow +1 \rightarrow -1 \rightarrow +1$, so it has **three monotone branches**.

Composing the two networks



(a) f_1 , a zigzag. (b) f_2 , over the range that f_1 produces. (c) the composition $f_2 \circ f_1$: f_2 's shape appears once per branch of f_1 .

- Six hidden units in total, but the composition has far more joints than any six-unit shallow network could

Why the pieces multiply

Proposition 1 (Composition of piecewise-linear maps)

*If $f_1 : \mathbb{R} \rightarrow \mathbb{R}$ is continuous and piecewise linear with p pieces, and f_2 with q pieces, then $f_2 \circ f_1$ is continuous and piecewise linear with **at most pq** pieces. The bound is attained when f_1 maps each of its pieces onto the whole domain of f_2 .*

- Let I be one of the p intervals on which f_1 is affine. On I , $f_2 \circ f_1$ can change slope only where $f_1(x)$ crosses a breakpoint of f_2
- f_1 is affine on I , so it crosses each of the $q - 1$ breakpoints of f_2 **at most once** — giving at most $q - 1$ new breakpoints, hence at most q pieces on I
- Summing over the p intervals gives at most pq pieces. Continuity is preserved because a composition of continuous maps is continuous □
- Equality needs every piece of f_1 to sweep all of f_2 's breakpoints — which is exactly what a **fold** does

Why the pieces multiply

Proposition 1 (Composition of piecewise-linear maps)

If $f_1 : \mathbb{R} \rightarrow \mathbb{R}$ is continuous and piecewise linear with p pieces, and f_2 with q pieces, then $f_2 \circ f_1$ is continuous and piecewise linear with **at most pq** pieces. The bound is attained when f_1 maps each of its pieces onto the whole domain of f_2 .

- Let I be one of the p intervals on which f_1 is affine. On I , $f_2 \circ f_1$ can change slope only where $f_1(x)$ crosses a breakpoint of f_2
- **f_1 is affine on I , so it crosses each of the $q - 1$ breakpoints of f_2 at most once — giving at most $q - 1$ new breakpoints, hence at most q pieces on I**
- Summing over the p intervals gives at most pq pieces. Continuity is preserved because a composition of continuous maps is continuous □
- Equality needs every piece of f_1 to sweep all of f_2 's breakpoints — which is exactly what a **fold** does

Why the pieces multiply

Proposition 1 (Composition of piecewise-linear maps)

If $f_1 : \mathbb{R} \rightarrow \mathbb{R}$ is continuous and piecewise linear with p pieces, and f_2 with q pieces, then $f_2 \circ f_1$ is continuous and piecewise linear with **at most pq** pieces. The bound is attained when f_1 maps each of its pieces onto the whole domain of f_2 .

- Let I be one of the p intervals on which f_1 is affine. On I , $f_2 \circ f_1$ can change slope only where $f_1(x)$ crosses a breakpoint of f_2
- f_1 is affine on I , so it crosses each of the $q - 1$ breakpoints of f_2 **at most once** — giving at most $q - 1$ new breakpoints, hence at most q pieces on I
- **Summing over the p intervals gives at most pq pieces. Continuity is preserved because a composition of continuous maps is continuous** □
- Equality needs every piece of f_1 to sweep all of f_2 's breakpoints — which is exactly what a **fold** does

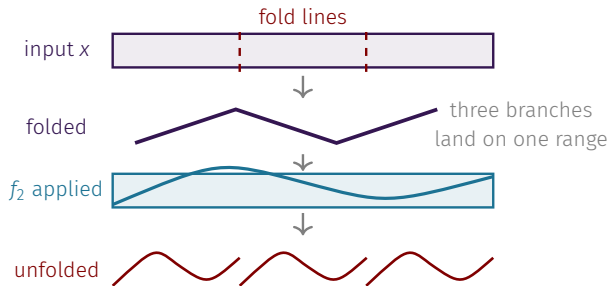
Why the pieces multiply

Proposition 1 (Composition of piecewise-linear maps)

If $f_1 : \mathbb{R} \rightarrow \mathbb{R}$ is continuous and piecewise linear with p pieces, and f_2 with q pieces, then $f_2 \circ f_1$ is continuous and piecewise linear with **at most pq** pieces. The bound is attained when f_1 maps each of its pieces onto the whole domain of f_2 .

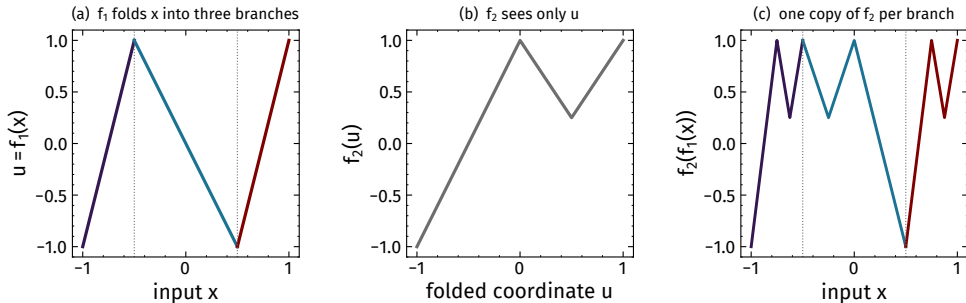
- Let I be one of the p intervals on which f_1 is affine. On I , $f_2 \circ f_1$ can change slope only where $f_1(x)$ crosses a breakpoint of f_2
- f_1 is affine on I , so it crosses each of the $q - 1$ breakpoints of f_2 **at most once** — giving at most $q - 1$ new breakpoints, hence at most q pieces on I
- Summing over the p intervals gives at most pq pieces. Continuity is preserved because a composition of continuous maps is continuous □
- **Equality needs every piece of f_1 to sweep all of f_2 's breakpoints — which is exactly what a fold does**

Folding: why the pattern repeats



- f_1 **folds** the input line: three different x land on the same u
- f_2 cannot tell them apart — so its shape is stamped **once per branch**

The same thing, in the plots



(a) the three branches of f_1 , coloured. (b) f_2 on the folded coordinate u . (c) the composition — each branch carries its own copy of f_2 , in the matching colour.

SHALLOW joints add: M units $\rightarrow M+1$ pieces

DEEP folding multiplies pieces across layers

Counting for our two networks

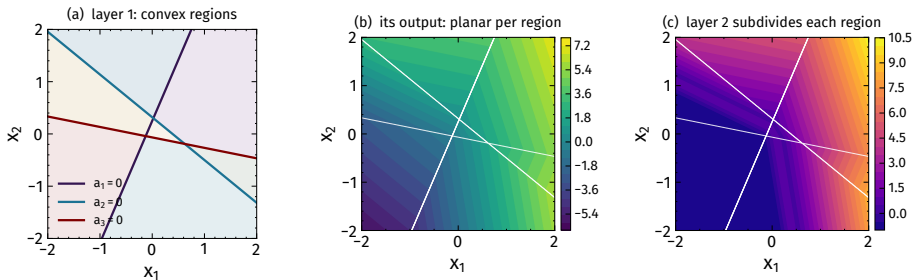
- f_1 has 3 hidden units, so 4 pieces and 3 monotone branches on $[-1, 1]$
- f_2 has 3 hidden units, so 4 pieces on its input range
- By the Proposition the composition has up to $3 \times 4 = 12$ pieces — from only 6 units

In general, for L layers of M units on a scalar input:

$$\underbrace{ML + 1}_{\text{shallow, } ML \text{ units}} \quad \text{versus} \quad \underbrace{(M + 1)^L}_{\text{deep, same unit count}}$$

Width adds pieces. Depth multiplies them.

Composing in two dimensions

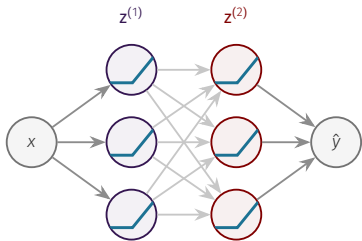


Layer 1 takes $\mathbf{x} = (x_1, x_2)$ into three ReLU units. (a) each unit contributes a line $a_j = 0$; together they cut the plane into *convex* regions, one per activation pattern. (b) the layer output is planar on each region. (c) layer 2 bends the values *within* every region, subdividing each one.

- In 2-D the fold becomes a **crease pattern**: depth creases an already-creased plane

The Deep Network

Two hidden layers



Each circle carries the activation h ; arrows are weights.

Layer 1, layer 2, then the read-out:

$$\mathbf{z}^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{z}^{(2)} = h(\mathbf{W}^{(2)}\mathbf{z}^{(1)} + \mathbf{b}^{(2)})$$

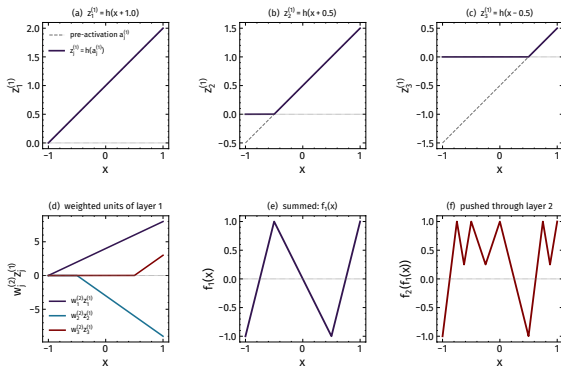
$$\hat{y} = \mathbf{W}^{(3)}\mathbf{z}^{(2)} + \mathbf{b}^{(3)}$$

Each layer: a **linear map** then an **activation**. Our $f_2 \circ f_1$ is exactly this, with $h = \text{ReLU}$.

SHALLOW one hidden layer

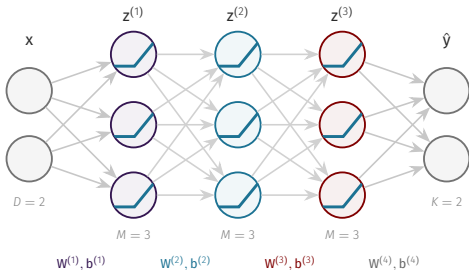
DEEP hidden layers *stacked*: the same block, composed

Computed step by step



(a–c) each unit of layer 1: dashed $a_j^{(1)}$, solid $z_j^{(1)}$. (d) scaled by the second-layer weights. (e) summed: f_1 . (f) pushed through layer 2.

The general deep network



$$z^{(l)} = h(W^{(l)}z^{(l-1)} + b^{(l)})$$

- $z^{(0)} = x$, and h acts element-wise
- Depth L = hidden layers (3 here); width M = units per layer — both **hyperparameters**
- Parameters here: $9 + 12 + 12 + 8 = 41$

Building depth recursively

The two-layer construction **extends recursively** (Bishop & Bishop, 2024, §6.2):

$$\mathbf{z}^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x}), \quad \mathbf{z}^{(2)} = h(\mathbf{W}^{(2)}\mathbf{z}^{(1)}), \quad \dots, \quad \hat{\mathbf{y}} = \mathbf{W}^{(L+1)}\mathbf{z}^{(L)}$$

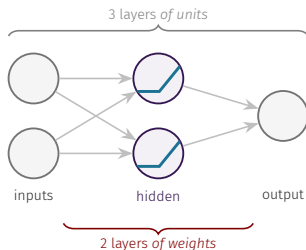
(biases absorbed by a clamped unit $z_0^{(l)} = 1$)

- Each layer feeds the next — the **same operation** (linear map, then nonlinearity) repeated
- Nothing new is needed: depth is **repetition of a single idea**

SHALLOW $\hat{\mathbf{y}} = \mathbf{W}^{(2)}h(\mathbf{W}^{(1)}\mathbf{x})$

DEEP $\hat{\mathbf{y}} = \mathbf{W}^{(L+1)}h(\dots h(\mathbf{W}^{(1)}\mathbf{x}))$

A note on counting layers



- The same network is called “three-layer”, “single-hidden-layer” or “two-layer” depending on what is counted
- Bishop counts **layers of weights** — the adaptive parameters B&B (2024), §6.3
- In this course: **deep** means more than one hidden layer, i.e. $L > 1$

Universal Approximation

What we already proved, and what is missing

Lemma 2 (One dimension, from Class 5)

*For every continuous f on $[a, b]$ and every $\varepsilon > 0$ there is a **one-hidden-layer** ReLU network $\hat{y}$ with finitely many units such that $\sup_{x \in [a, b]} |f(x) - \hat{y}(x)| < \varepsilon$.*

- We proved this **constructively**: two ReLUs make a ramp, slope increments reproduce the piecewise-linear interpolant exactly, uniform continuity bounds the error
- What is missing is the general case: D inputs, and an arbitrary non-polynomial activation

The route to D dimensions is to reduce it to the one-dimensional case, using functions that vary along a **single direction**.

Ridge functions

A **ridge function** is one that depends on $\mathbf{x}$ only through a single linear combination:

$$\mathbf{x} \mapsto g(\mathbf{w}^\top \mathbf{x} + b), \quad \mathbf{w} \in \mathbb{R}^D, b \in \mathbb{R}$$

- It is **constant** on every hyperplane perpendicular to $\mathbf{w}$ — all of its variation is along one direction
- Every hidden unit of a network is a ridge function: $h(\mathbf{w}_j^\top \mathbf{x} + w_{j0})$
- So a one-hidden-layer network is exactly a **finite sum of ridge functions**

If sums of ridge functions reach any continuous function, and Lemma 1 reproduces any *one-dimensional* profile, we are done.

Ridge functions are enough

Lemma 3 (Density of trigonometric ridge functions)

Let $\mathcal{K} \subset \mathbb{R}^D$ be compact. The set

$$\mathcal{R} = \left\{ \mathbf{x} \mapsto \sum_{k=1}^K c_k \cos(\mathbf{w}_k^\top \mathbf{x} + b_k) : K \in \mathbb{N}, c_k, b_k \in \mathbb{R}, \mathbf{w}_k \in \mathbb{R}^D \right\}$$

is dense in $C(\mathcal{K})$ under the supremum norm.

Proof (Stone–Weierstrass; check the three hypotheses).

- **Algebra.** $\mathcal{R}$ is closed under products, because $\cos A \cos B = \frac{1}{2} [\cos(A - B) + \cos(A + B)]$ turns a product of two ridge terms into a sum of two more
- **Constants.** Take $\mathbf{w} = \mathbf{0}$, $b = 0$: the constant function 1 lies in $\mathcal{R}$
- **Separates points.** For $\mathbf{x} \neq \mathbf{y}$ choose $\mathbf{w}$ with $t = \mathbf{w}^\top (\mathbf{x} - \mathbf{y}) \in (0, \pi)$; then $\cos(\mathbf{w}^\top \mathbf{x}) \neq \cos(\mathbf{w}^\top \mathbf{y})$ for a suitable offset
- Stone–Weierstrass then gives density in $C(\mathcal{K})$

□

Ridge functions are enough

Lemma 3 (Density of trigonometric ridge functions)

Let $\mathcal{K} \subset \mathbb{R}^D$ be compact. The set

$$\mathcal{R} = \left\{ \mathbf{x} \mapsto \sum_{k=1}^K c_k \cos(\mathbf{w}_k^\top \mathbf{x} + b_k) : K \in \mathbb{N}, c_k, b_k \in \mathbb{R}, \mathbf{w}_k \in \mathbb{R}^D \right\}$$

is dense in $C(\mathcal{K})$ under the supremum norm.

Proof (Stone–Weierstrass; check the three hypotheses).

- **Algebra.** $\mathcal{R}$ is closed under products, because $\cos A \cos B = \frac{1}{2} [\cos(A - B) + \cos(A + B)]$ turns a product of two ridge terms into a sum of two more
- **Constants.** Take $\mathbf{w} = \mathbf{0}$, $b = 0$: the constant function 1 lies in $\mathcal{R}$
- **Separates points.** For $\mathbf{x} \neq \mathbf{y}$ choose $\mathbf{w}$ with $t = \mathbf{w}^\top (\mathbf{x} - \mathbf{y}) \in (0, \pi)$; then $\cos(\mathbf{w}^\top \mathbf{x}) \neq \cos(\mathbf{w}^\top \mathbf{y})$ for a suitable offset
- Stone–Weierstrass then gives density in $C(\mathcal{K})$ □

Ridge functions are enough

Lemma 3 (Density of trigonometric ridge functions)

Let $\mathcal{K} \subset \mathbb{R}^D$ be compact. The set

$$\mathcal{R} = \left\{ \mathbf{x} \mapsto \sum_{k=1}^K c_k \cos(\mathbf{w}_k^\top \mathbf{x} + b_k) : K \in \mathbb{N}, c_k, b_k \in \mathbb{R}, \mathbf{w}_k \in \mathbb{R}^D \right\}$$

is dense in $C(\mathcal{K})$ under the supremum norm.

Proof (Stone–Weierstrass; check the three hypotheses).

- **Algebra.** $\mathcal{R}$ is closed under products, because $\cos A \cos B = \frac{1}{2} [\cos(A - B) + \cos(A + B)]$ turns a product of two ridge terms into a sum of two more
- **Constants.** Take $\mathbf{w} = \mathbf{0}$, $b = 0$: the constant function 1 lies in $\mathcal{R}$
- **Separates points.** For $\mathbf{x} \neq \mathbf{y}$ choose $\mathbf{w}$ with $t = \mathbf{w}^\top (\mathbf{x} - \mathbf{y}) \in (0, \pi)$; then $\cos(\mathbf{w}^\top \mathbf{x}) \neq \cos(\mathbf{w}^\top \mathbf{y})$ for a suitable offset
- Stone–Weierstrass then gives density in $C(\mathcal{K})$

□

Ridge functions are enough

Lemma 3 (Density of trigonometric ridge functions)

Let $\mathcal{K} \subset \mathbb{R}^D$ be compact. The set

$$\mathcal{R} = \left\{ \mathbf{x} \mapsto \sum_{k=1}^K c_k \cos(\mathbf{w}_k^\top \mathbf{x} + b_k) : K \in \mathbb{N}, c_k, b_k \in \mathbb{R}, \mathbf{w}_k \in \mathbb{R}^D \right\}$$

is dense in $C(\mathcal{K})$ under the supremum norm.

Proof (Stone–Weierstrass; check the three hypotheses).

- **Algebra.** $\mathcal{R}$ is closed under products, because $\cos A \cos B = \frac{1}{2} [\cos(A - B) + \cos(A + B)]$ turns a product of two ridge terms into a sum of two more
- **Constants.** Take $\mathbf{w} = \mathbf{0}$, $b = 0$: the constant function 1 lies in $\mathcal{R}$
- **Separates points.** For $\mathbf{x} \neq \mathbf{y}$ choose $\mathbf{w}$ with $t = \mathbf{w}^\top (\mathbf{x} - \mathbf{y}) \in (0, \pi)$; then $\cos(\mathbf{w}^\top \mathbf{x}) \neq \cos(\mathbf{w}^\top \mathbf{y})$ for a suitable offset
- **Stone–Weierstrass then gives density in $C(\mathcal{K})$**

□

Universal approximation in D dimensions

Theorem 4 (Cybenko 1989; Hornik 1991)

Let h be continuous and *not a polynomial*, $\mathcal{K} \subset \mathbb{R}^D$ compact and $f \in C(\mathcal{K})$. For every $\varepsilon > 0$ there exist $M \in \mathbb{N}$ and weights such that

$$\hat{y}(\mathbf{x}) = w_0^{(2)} + \sum_{j=1}^M w_j^{(2)} h(\mathbf{w}_j^{(1)\top} \mathbf{x} + w_{j0}^{(1)}) \quad \text{satisfies} \quad \sup_{\mathbf{x} \in \mathcal{K}} |f(\mathbf{x}) - \hat{y}(\mathbf{x})| < \varepsilon.$$

- **Step 1.** By Lemma 2 pick $g = \sum_{k=1}^K c_k \cos(\mathbf{w}_k^\top \mathbf{x} + b_k)$ with $\sup_{\mathcal{K}} |f - g| < \varepsilon/2$
- **Step 2.** As $\mathbf{x}$ ranges over the compact $\mathcal{K}$, each $t_k = \mathbf{w}_k^\top \mathbf{x} + b_k$ ranges over a compact interval I_k
- **Step 3.** By Lemma 1 approximate $\cos$ on I_k by $\sum_j \alpha_{kj} h(\beta_{kj} t + \gamma_{kj})$ to within $\varepsilon/(2K|c_k|)$
- **Step 4.** Substitute $t = \mathbf{w}_k^\top \mathbf{x} + b_k$. Then $h(\beta_{kj} t + \gamma_{kj}) = h(\tilde{\mathbf{w}}_k^\top \mathbf{x} + \tilde{w}_0)$ – still **one hidden layer**.
Summing over k and applying the triangle inequality gives $\varepsilon/2 + \varepsilon/2$

□

Universal approximation in D dimensions

Theorem 4 (Cybenko 1989; Hornik 1991)

Let h be continuous and *not a polynomial*, $\mathcal{K} \subset \mathbb{R}^D$ compact and $f \in C(\mathcal{K})$. For every $\varepsilon > 0$ there exist $M \in \mathbb{N}$ and weights such that

$$\hat{y}(\mathbf{x}) = w_0^{(2)} + \sum_{j=1}^M w_j^{(2)} h(\mathbf{w}_j^{(1)\top} \mathbf{x} + w_{j0}^{(1)}) \quad \text{satisfies} \quad \sup_{\mathbf{x} \in \mathcal{K}} |f(\mathbf{x}) - \hat{y}(\mathbf{x})| < \varepsilon.$$

- Step 1. By Lemma 2 pick $g = \sum_{k=1}^K c_k \cos(\mathbf{w}_k^\top \mathbf{x} + b_k)$ with $\sup_{\mathcal{K}} |f - g| < \varepsilon/2$
- Step 2. As $\mathbf{x}$ ranges over the compact $\mathcal{K}$, each $t_k = \mathbf{w}_k^\top \mathbf{x} + b_k$ ranges over a compact interval I_k
- Step 3. By Lemma 1 approximate $\cos$ on I_k by $\sum_j \alpha_{kj} h(\beta_{kj} t + \gamma_{kj})$ to within $\varepsilon/(2K|c_k|)$
- Step 4. Substitute $t = \mathbf{w}_k^\top \mathbf{x} + b_k$. Then $h(\beta_{kj} t + \gamma_{kj}) = h(\tilde{\mathbf{w}}_k^\top \mathbf{x} + \tilde{w}_0)$ – still *one hidden layer*.
Summing over k and applying the triangle inequality gives $\varepsilon/2 + \varepsilon/2$

□

Universal approximation in D dimensions

Theorem 4 (Cybenko 1989; Hornik 1991)

Let h be continuous and *not a polynomial*, $\mathcal{K} \subset \mathbb{R}^D$ compact and $f \in C(\mathcal{K})$. For every $\varepsilon > 0$ there exist $M \in \mathbb{N}$ and weights such that

$$\hat{y}(\mathbf{x}) = w_0^{(2)} + \sum_{j=1}^M w_j^{(2)} h(\mathbf{w}_j^{(1)\top} \mathbf{x} + w_{j0}^{(1)}) \quad \text{satisfies} \quad \sup_{\mathbf{x} \in \mathcal{K}} |f(\mathbf{x}) - \hat{y}(\mathbf{x})| < \varepsilon.$$

- Step 1. By Lemma 2 pick $g = \sum_{k=1}^K c_k \cos(\mathbf{w}_k^\top \mathbf{x} + b_k)$ with $\sup_{\mathcal{K}} |f - g| < \varepsilon/2$
- Step 2. As $\mathbf{x}$ ranges over the compact $\mathcal{K}$, each $t_k = \mathbf{w}_k^\top \mathbf{x} + b_k$ ranges over a compact interval I_k
- Step 3. By Lemma 1 approximate $\cos$ on I_k by $\sum_j \alpha_{kj} h(\beta_{kj} t + \gamma_{kj})$ to within $\varepsilon/(2K|c_k|)$
- Step 4. Substitute $t = \mathbf{w}_k^\top \mathbf{x} + b_k$. Then $h(\beta_{kj} t + \gamma_{kj}) = h(\tilde{\mathbf{w}}_k^\top \mathbf{x} + \tilde{w}_0)$ – still one hidden layer. Summing over k and applying the triangle inequality gives $\varepsilon/2 + \varepsilon/2$

□

Universal approximation in D dimensions

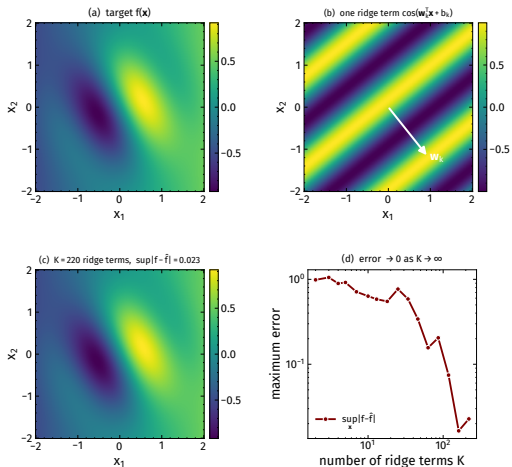
Theorem 4 (Cybenko 1989; Hornik 1991)

Let h be continuous and *not a polynomial*, $\mathcal{K} \subset \mathbb{R}^D$ compact and $f \in C(\mathcal{K})$. For every $\varepsilon > 0$ there exist $M \in \mathbb{N}$ and weights such that

$$\hat{y}(\mathbf{x}) = w_0^{(2)} + \sum_{j=1}^M w_j^{(2)} h(\mathbf{w}_j^{(1)\top} \mathbf{x} + w_{j0}^{(1)}) \quad \text{satisfies} \quad \sup_{\mathbf{x} \in \mathcal{K}} |f(\mathbf{x}) - \hat{y}(\mathbf{x})| < \varepsilon.$$

- Step 1. By Lemma 2 pick $g = \sum_{k=1}^K c_k \cos(\mathbf{w}_k^\top \mathbf{x} + b_k)$ with $\sup_{\mathcal{K}} |f - g| < \varepsilon/2$
- Step 2. As $\mathbf{x}$ ranges over the compact $\mathcal{K}$, each $t_k = \mathbf{w}_k^\top \mathbf{x} + b_k$ ranges over a compact interval I_k
- Step 3. By Lemma 1 approximate $\cos$ on I_k by $\sum_j \alpha_{kj} h(\beta_{kj} t + \gamma_{kj})$ to within $\varepsilon/(2K|c_k|)$
- Step 4. Substitute $t = \mathbf{w}_k^\top \mathbf{x} + b_k$. Then $h(\beta_{kj} t + \gamma_{kj}) = h(\tilde{\mathbf{w}}^\top \mathbf{x} + \tilde{w}_0)$ — still one hidden layer. Summing over k and applying the triangle inequality gives $\varepsilon/2 + \varepsilon/2$ □

The construction, drawn



- (a) a continuous target on a compact square
- (b) *one* ridge term: constant along the directions perpendicular to $\mathbf{w}_k$
- (c) $K = 220$ such terms, fitted by least squares — already indistinguishable from the target
- (d) the error falls steadily as K grows, which is Step 1 of the proof in action

Which activations work?

Theorem 5 (Leshno, Lin, Pinkus & Schocken, 1993)

For continuous h , one-hidden-layer networks are dense in $C(\mathcal{K})$ for every compact $\mathcal{K}$ if and only if h is not a polynomial.

Proof of the “only if” direction. Suppose h is a polynomial of degree p . Then $h(\mathbf{w}^\top \mathbf{x} + w_0)$ is a polynomial in $\mathbf{x}$ of degree at most p , and any finite sum of such terms is too. So every network output lies in the finite-dimensional space of polynomials of degree $\leq p$, which is closed and is not all of $C(\mathcal{K})$. No amount of width escapes it. $\square$

- This is exactly the Class 5 observation that without a nonlinearity the network collapses to a line, in its sharpest form
- ReLU, tanh, sigmoid, softplus and GELU all qualify; $h(a) = a^2$ does not

What ReLU networks compute, exactly

Proposition 6 (ReLU networks are the piecewise-linear functions)

(i) Every ReLU network computes a *continuous piecewise-linear* function. (ii) Conversely, every continuous piecewise-linear $f: \mathbb{R}^D \rightarrow \mathbb{R}$ is computed *exactly* by a ReLU network with at most $\lceil \log_2(D+1) \rceil$ hidden layers. (ii): Arora et al., 2018

Proof of (i), by induction on the layer index. The identity map is piecewise linear. If $\mathbf{z}^{(l-1)}$ is piecewise linear then so is $\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}$, since affine maps and sums of piecewise-linear functions are piecewise linear; and $h = \max(0, \cdot)$ is itself piecewise linear, so the composition is too. Continuity is preserved at every step. □

Depth does *not* enlarge the class a ReLU network can represent — shallow and deep both give exactly the continuous piecewise-linear functions. What changes is *how many units* it costs.

What universality does *not* settle

- It is an **existence result**. Good weights exist; it says nothing about whether gradient descent finds them
- It says nothing about **generalisation**. Fitting a training set is not the goal
- It gives no useful **size bound**. The grid argument in D dimensions needs $M = O((L_f/\epsilon)^D)$ units
- Both **shallow and deep networks are universal**, so universality cannot be the reason to prefer depth

The right question is not *whether* a network can represent a function, but **how efficiently**.

Depth versus Width

A function that depth computes cheaply

Lemma 7 (The sawtooth)

Let $T(x) = 2h(x) - 4h(x - \frac{1}{2})$, a *two-unit* ReLU network. Then T maps $[0, 1]$ onto $[0, 1]$, folding it in half, and the L -fold composition $T^{\circ L}$ is piecewise linear with *exactly* 2^L *pieces*, computed by a ReLU network of depth L and width 2.

- $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $T(x) = 2 - 2x$ on $[\frac{1}{2}, 1]$: each half is mapped affinely onto all of $[0, 1]$
- Base case. $T^{\circ 1} = T$ has $2^1 = 2$ pieces
- Induction. Write $T^{\circ L} = T^{\circ(L-1)} \circ T$. On each half of $[0, 1]$, T is affine and onto, so $T^{\circ L}$ restricted there is a rescaled copy of $T^{\circ(L-1)}$, contributing 2^{L-1} pieces
- Two halves give $2 \cdot 2^{L-1} = 2^L$ pieces



A function that depth computes cheaply

Lemma 7 (The sawtooth)

Let $T(x) = 2h(x) - 4h(x - \frac{1}{2})$, a **two-unit** ReLU network. Then T maps $[0, 1]$ onto $[0, 1]$, folding it in half, and the L -fold composition $T^{\circ L}$ is piecewise linear with **exactly 2^L pieces**, computed by a ReLU network of depth L and width 2.

- $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $T(x) = 2 - 2x$ on $[\frac{1}{2}, 1]$: each half is mapped **affinely onto all of $[0, 1]$**
- **Base case.** $T^{\circ 1} = T$ has $2^1 = 2$ pieces
- **Induction.** Write $T^{\circ L} = T^{\circ(L-1)} \circ T$. On each half of $[0, 1]$, T is affine and onto, so $T^{\circ L}$ restricted there is a rescaled copy of $T^{\circ(L-1)}$, contributing 2^{L-1} pieces
- Two halves give $2 \cdot 2^{L-1} = 2^L$ pieces

□

A function that depth computes cheaply

Lemma 7 (The sawtooth)

Let $T(x) = 2h(x) - 4h(x - \frac{1}{2})$, a **two-unit** ReLU network. Then T maps $[0, 1]$ onto $[0, 1]$, folding it in half, and the L -fold composition $T^{\circ L}$ is piecewise linear with **exactly 2^L pieces**, computed by a ReLU network of depth L and width 2.

- $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $T(x) = 2 - 2x$ on $[\frac{1}{2}, 1]$: each half is mapped **affinely onto all of $[0, 1]$**
- **Base case.** $T^{\circ 1} = T$ has $2^1 = 2$ pieces
- **Induction.** Write $T^{\circ L} = T^{\circ(L-1)} \circ T$. On each half of $[0, 1]$, T is affine and onto, so $T^{\circ L}$ restricted there is a **rescaled copy of $T^{\circ(L-1)}$, contributing 2^{L-1} pieces**
- Two halves give $2 \cdot 2^{L-1} = 2^L$ pieces



A function that depth computes cheaply

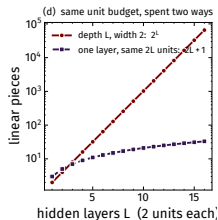
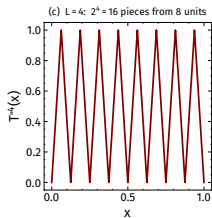
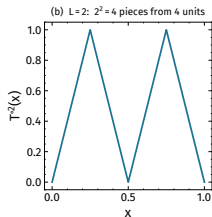
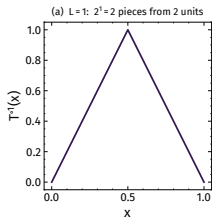
Lemma 7 (The sawtooth)

Let $T(x) = 2h(x) - 4h(x - \frac{1}{2})$, a **two-unit** ReLU network. Then T maps $[0, 1]$ onto $[0, 1]$, folding it in half, and the L -fold composition $T^{\circ L}$ is piecewise linear with **exactly 2^L pieces**, computed by a ReLU network of depth L and width 2.

- $T(x) = 2x$ on $[0, \frac{1}{2}]$ and $T(x) = 2 - 2x$ on $[\frac{1}{2}, 1]$: each half is mapped **affinely onto all of $[0, 1]$**
- **Base case.** $T^{\circ 1} = T$ has $2^1 = 2$ pieces
- **Induction.** Write $T^{\circ L} = T^{\circ(L-1)} \circ T$. On each half of $[0, 1]$, T is affine and onto, so $T^{\circ L}$ restricted there is a rescaled copy of $T^{\circ(L-1)}$, contributing 2^{L-1} pieces
- **Two halves give $2 \cdot 2^{L-1} = 2^L$ pieces**



The sawtooth, drawn



- (a–c) each extra layer **doubles** the number of teeth, at a cost of two more units
- (d) the same total budget of $2L$ units: spent as depth it buys 2^L pieces, spent as width only $2L + 1$
- By $L = 16$ that is 65,536 against 33

Depth beats width, provably

Theorem 8 (Depth–width separation; after Telgarsky, 2016)

For every $L \in \mathbb{N}$ the function $T^{\circ L}$ is computed by a ReLU network with L hidden layers of width 2, that is $O(L)$ parameters. Any *one-hidden-layer* ReLU network computing $T^{\circ L}$ exactly needs at least $2^L - 1$ units.

- From Class 5: a one-hidden-layer ReLU network with M units on a scalar input is piecewise linear with at most $M + 1$ pieces
- By the Lemma, $T^{\circ L}$ has exactly 2^L pieces
- Matching them requires $M + 1 \geq 2^L$, hence $M \geq 2^L - 1$ □
- Telgarsky's full theorem strengthens this from *exact* representation to *approximation*: no shallow network of sub-exponential size can even come close

An **exponential** gap, from a function you can draw in one line.

Depth beats width, provably

Theorem 8 (Depth–width separation; after Telgarsky, 2016)

For every $L \in \mathbb{N}$ the function $T^{\circ L}$ is computed by a ReLU network with L hidden layers of width 2, that is $O(L)$ parameters. Any *one-hidden-layer* ReLU network computing $T^{\circ L}$ exactly needs at least $2^L - 1$ units.

- From Class 5: a one-hidden-layer ReLU network with M units on a scalar input is piecewise linear with at most $M + 1$ pieces
- **By the Lemma, $T^{\circ L}$ has exactly 2^L pieces**
- Matching them requires $M + 1 \geq 2^L$, hence $M \geq 2^L - 1$ □
- Telgarsky's full theorem strengthens this from *exact* representation to *approximation*: no shallow network of sub-exponential size can even come close

An **exponential** gap, from a function you can draw in one line.

Depth beats width, provably

Theorem 8 (Depth–width separation; after Telgarsky, 2016)

For every $L \in \mathbb{N}$ the function $T^{\circ L}$ is computed by a ReLU network with L hidden layers of width 2, that is $O(L)$ parameters. Any *one-hidden-layer* ReLU network computing $T^{\circ L}$ exactly needs at least $2^L - 1$ units.

- From Class 5: a one-hidden-layer ReLU network with M units on a scalar input is piecewise linear with at most $M + 1$ pieces
- By the Lemma, $T^{\circ L}$ has exactly 2^L pieces
- **Matching them requires $M + 1 \geq 2^L$, hence $M \geq 2^L - 1$** □
- Telgarsky's full theorem strengthens this from *exact* representation to *approximation*: no shallow network of sub-exponential size can even come close

An **exponential** gap, from a function you can draw in one line.

Depth beats width, provably

Theorem 8 (Depth–width separation; after Telgarsky, 2016)

For every $L \in \mathbb{N}$ the function $T^{\circ L}$ is computed by a ReLU network with L hidden layers of width 2, that is $O(L)$ parameters. Any *one-hidden-layer* ReLU network computing $T^{\circ L}$ exactly needs at least $2^L - 1$ units.

- From Class 5: a one-hidden-layer ReLU network with M units on a scalar input is piecewise linear with at most $M + 1$ pieces
- By the Lemma, $T^{\circ L}$ has exactly 2^L pieces
- Matching them requires $M + 1 \geq 2^L$, hence $M \geq 2^L - 1$ □
- Telgarsky's full theorem strengthens this from *exact representation* to *approximation*: no shallow network of sub-exponential size can even come close

An **exponential** gap, from a function you can draw in one line.

The same effect in D dimensions

Theorem 9 (Region lower bound; Montúfar et al., 2014)

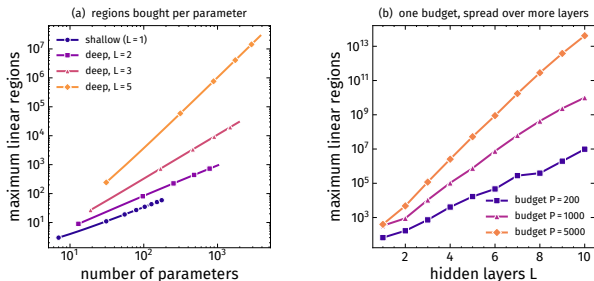
A ReLU network with L hidden layers of width $M \geq D$ on $\mathbb{R}^D$ can realise at least

$$\left(\prod_{l=1}^{L-1} \lfloor M/D \rfloor^D \right) \sum_{j=0}^D \binom{M}{j}$$

linear regions — *exponential in the depth L , but only polynomial in the width M .*

- The mechanism is the folding argument, iterated: each layer folds the already-folded input space
- The trailing $\sum_{j \leq D} \binom{M}{j}$ is exactly the **Zaslavsky count** from Class 5 — the last layer behaves like a shallow network
- Lower bound only: it says such networks *exist*, not that training finds them

Linear regions per parameter



Counts for a 1-input, 1-output ReLU network: $P_{\text{shallow}}(M) = 3M + 1$ and

$P_{\text{deep}}(M, L) = 2M + (L-1)M(M+1) + M + 1$, with $M+1$ and $(M+1)^L$ regions. (a) regions bought per parameter, log-log. (b) a fixed budget P : for each L we take the widest M that still fits, then plot the regions it buys.

SHALLOW regions grow *polynomially* with width

DEEP regions grow *exponentially* with depth

Shallow vs. deep: the trade-off

	Shallow ($L = 1$)	Deep ($L > 1$)
Function class	continuous piecewise linear	the same class
Expressivity	universal, but may need 2^L units	universal with $O(L)$ units
Regions	polynomial in width	exponential in depth
Features	one level of learned features	hierarchical features, layer on layer
Training	convex read-out only	harder landscape; needs back-propagation

Depth trades a harder optimisation problem for a dramatic gain in representational efficiency.

What Deep Networks Represent

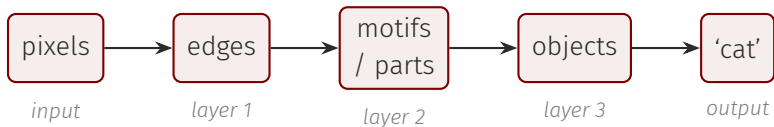
Depth as an inductive bias

- Universality tells us what a network *can* represent; it says nothing about what it *prefers* to learn
- A deep architecture encodes an **inductive bias**: that the output relates to the input through a **hierarchy** of intermediate concepts B&B, §6.3.1
- For images, the map from pixels to a concept like ‘cat’ is **highly nonlinear** — hopeless for one layer, natural for many
- The right bias means the network needs **less data** and generalises better

SHALLOW bias: the output is one nonlinear map of the input

DEEP bias: the output is a **hierarchy** of simpler maps

Hierarchical representations



- Early layers detect **low-level** features; later layers assemble them into **high-level** concepts
 - Each layer reuses the layer below — features are **shared** by everything built on top
- B&B, §6.3.1

Why hierarchy is the right bias for real data

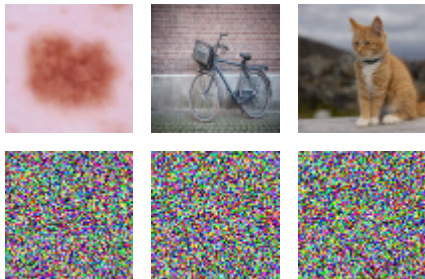
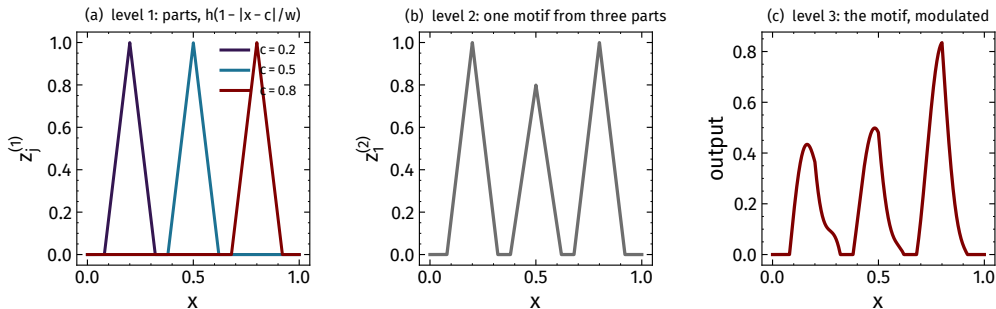


Figure from Bishop & Bishop (2024), Fig. 6.8:
natural images (top) vs. random pixels (bottom).

- Almost every random pixel pattern is **noise** — images like the one above are vanishingly rare among all possible images
- Natural data lives on a **low-dimensional manifold** with strong structure
- That structure is **compositional**, so a compositional model captures it efficiently

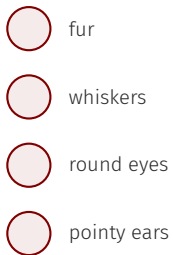
A hierarchy, built explicitly



(a) three level-1 units, each a bump $h(1 - |x - c|/w)$ centred at $c \in \{0.2, 0.5, 0.8\}$ with $w = 0.12$. (b) a level-2 unit sums them into one motif. (c) level 3 modulates that motif by $0.6 + 0.4 \sin(6\pi x)$.

- Each stage is a **simple** operation on the stage below — yet the result is intricate

Distributed representations



a hidden layer

- Each unit signals whether a **feature** is present (high) or absent (low)
 - M units name M features — but their **combinations** encode up to 2^M distinct patterns
 - Concepts are represented by **patterns of activity**, not single units — a **distributed code**
- B&B, §6.3.2

SHALLOW features fixed and hand-chosen

DEEP features learned, and **shared combinatorially**

Representation learning: the set-up

Two interleaved half-moons, 400 points, Gaussian noise $\sigma = 0.1$, inputs standardised. Network $2 \rightarrow 24 \rightarrow 2 \rightarrow 1$, all ReLU:

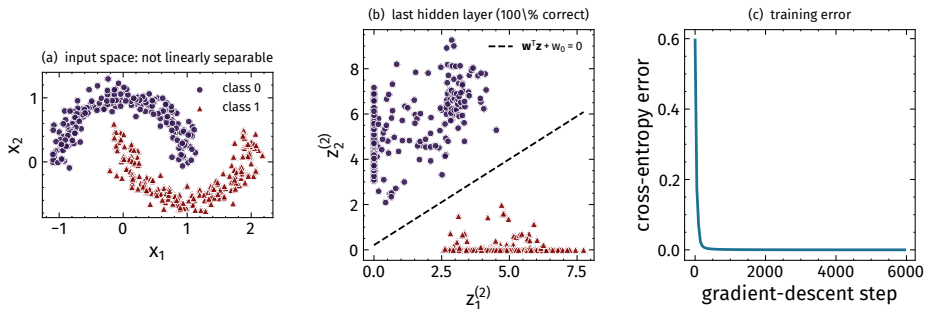
$$\mathbf{z}^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{z}^{(2)} = h(\mathbf{W}^{(2)}\mathbf{z}^{(1)} + \mathbf{b}^{(2)}), \quad a = \mathbf{w}^{(3)\top}\mathbf{z}^{(2)} + w_0^{(3)}$$

Trained by gradient descent on the binary cross-entropy of $\hat{y} = \sigma(a)$ – exactly the error function derived in Class 6:

$$E = -\frac{1}{N} \sum_n \left[y_n \ln \hat{y}_n + (1 - y_n) \ln(1 - \hat{y}_n) \right]$$

- Learning rate 0.5, 6000 full-batch steps, He initialisation
- The last hidden layer is **2-dimensional on purpose**, so we can plot the representation the network learns

Representation learning: the result



(a) the data in input space — no straight line separates the classes. (b) the same points in the coordinates $(z_1^{(2)}, z_2^{(2)})$ of the last hidden layer; dashed is the output layer's boundary. (c) the error over training.

- The network has **rearranged the data** until a line is enough

cf. B&B, §6.3.3

What that experiment shows

- Each layer **transforms** the data to make the task easier for the next one
- By the final hidden layer the classes are arranged so a **linear** classifier suffices — which is all the output layer ever was
- That layer's output is the **learned representation** (the *embedding*) B&B, §6.3.3
- Nobody designed those coordinates — they were **learned from the data**

SHALLOW one transform, then a linear read-out

DEEP many learned transforms, then a linear read-out

Why depth tends to win in practice

- Real data is **compositional**: edges $\rightarrow$ shapes $\rightarrow$ objects; phonemes $\rightarrow$ words $\rightarrow$ meaning
- Deep networks mirror this — each layer builds **features of the previous layer's features**
- Fewer parameters for the same function means **better generalisation** and less data needed
- Caveat: depth alone is not enough — training deep networks needs the tools of the coming classes (initialisation, normalisation, residual connections)

SHALLOW flat: features of the raw input

DEEP hierarchical: features of features of features

Takeaways

1. A **deep network** is shallow networks **composed**: each layer is a linear map followed by an activation
2. Composition **folds** the input, so pieces **multiply** rather than add — at most pq from p and q , which is why 6 units gave 12 pieces
3. **Universal approximation** holds in D dimensions for any non-polynomial activation, by reducing to the one-dimensional case through **ridge functions**
4. Universality does not favour depth: ReLU networks compute exactly the continuous piecewise-linear functions, shallow or deep. **Efficiency** does — the sawtooth needs $O(L)$ units deep and $2^L - 1$ shallow
5. Depth suits **compositional** data by building hierarchical, distributed representations

Next: how to actually **train** these networks — backpropagation and gradient-based optimisation.

References

- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Ch. 6 — recursive construction, layer counting, hierarchical and distributed representations, representation learning. Figure 6.8 reproduced with attribution.
- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Ch. 4 — composition, folding, region counting.
- G. Cybenko, *Math. Control Signals Systems* 2:303, 1989; K. Hornik, *Neural Networks* 4:251, 1991 — universal approximation.
- M. Leshno, V. Lin, A. Pinkus & S. Schocken, *Neural Networks* 6:861, 1993 — non-polynomial activations.
- M. Telgarsky, *COLT*, 2016 — depth-width separation; G. Montúfar et al., *NeurIPS*, 2014 — linear-region bounds; R. Arora et al., *ICLR*, 2018 — ReLU networks and piecewise-linear functions.
- All plots generated by `src/build_all.py`; one module per figure under `src/figures/`.

Questions?